Assignment 1
Parallel Solver for the Steady-State 2D Heat Distribution Problem

Objectives

This assignment covers shared-memory parallel programming (OpenMP), distributed-memory parallel programming (MPI), domain decomposition, stencil computations, and performance evaluation.

Students are provided with a sequential C implementation of a steady-state 2D heat distribution solver using the *finite difference* method (*Jacobi iteration*). Your task is to develop two parallel versions—one using OpenMP and one using MPI—and conduct a rigorous performance comparison between the three programs.

Problem Description and Mathematical Model

Heat Equation

The heat distribution problem is an important topic in science and engineering, and it plays a key role in areas such as climate modeling and weather forecasting. The two-dimensional (2D) heat distribution problem aims to find the steady-state temperature distribution within a square area where the edges are held at fixed temperatures (boundary conditions) and the interior settles into thermal equilibrium.

The temperature $u(x, y, t)$ at a point (x, y) over a unit area at time t can be modeled using the *heat equation*, a second-order partial differential equation (PDE), as follows:

$$\alpha \nabla^2 u = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = \frac{\partial u}{\partial t} \quad \ldots(1)$$

where $0 \leq x \leq 1$, $0 \leq y \leq 1$, and α is is the thermal diffusivity of the material.

Steady-state means $\frac{\partial u}{\partial t} = 0$, this reduces the equation to the Laplace equation:

$$\nabla^2 u = 0 \quad \ldots(2)$$

In this problem, we would set the initial and boundary conditions for the problem as follows:

Initial condition:

$$u(x, y, t) = 25, \text{ for } 0 < x < 1 \text{ and } 0 < y < 1.$$

Boundary conditions:

$$u(0, y, t) = 0, \quad u(x, 0, t) = 1000,$$
$$u(1, y, t) = 0, \quad u(x, 1, t) = 0.$$

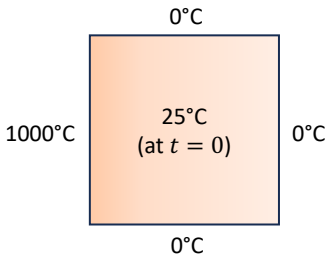


Figure 1. Initial and boundary conditions

You may imagine this is a top view of a square room which has a wall heater (at 1,000 °C) at one side and the other three sides are freezing glass windows (at 0 °C) as it is snowing outdoors. For convenience, the initial temperature over the entire area is set to room temperature 25 °C except the boundaries. You can expect the temperature at each point of the whole interior area will keep changing according to heat diffusion and finally stabilizes at some equilibrium since the boundary temperatures are fixed.

The Finite Difference Method

We will solve this problem numerically using the ***finite difference*** method. In this approach, we discretize space and time so that there is an integer number of spatial grid points and time steps at which we can calculate the solution. (If you are interested in why this method can approximate the steady-state solution, you may refer to this online reading.) In our case, the 2D space is divided evenly into a fine mesh of points. For each interior grid point (i, j), we perform a **stencil computation**: the temperature $u_{i,j}$ is updated using the average of the temperatures at its four neighboring points, as illustrated in the diagram below:

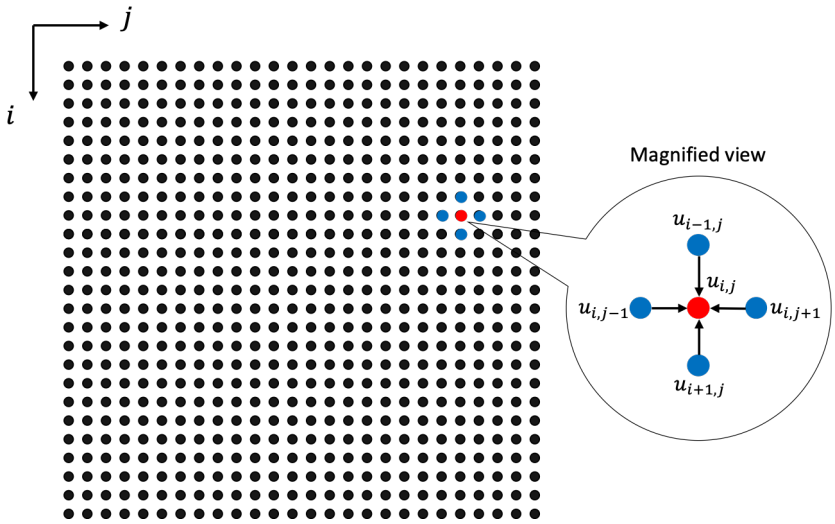


Figure 2. Schematic of the 5-point stencil for updating the interior grid value $u_{i,j}$ using its four nearest neighbors.

For an $n \times n$ grid, there are $(n-2) \times (n-2)$ interior points and the temperature at each of them can be computed by iterating the equation:

$$u_{i,j} = \frac{u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1}}{4} \quad \ldots(3)$$

for a fixed number of iterations or until the difference between iterations of a point is less than some predefined error tolerance ε (say 0.001).

The Jacobi Iterative Method

Equation (3) is essentially the *Gauss-Seidel iterative method*, which relies on a sequential order of computation and is therefore more difficult to parallelize. In this assignment, we adopt the *Jacobi method*:

$$w_{i,j} = \frac{u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1}}{4} \quad \ldots(4)$$

where w stores the temperatures computed in the current iteration, with each value computed from the four neighboring values stored in u from the previous iteration.

We can observe that equation (4) converges more slowly than equation (3) and requires more memory (two 2-D arrays w and u needed), but it enables easier parallelization.

Programming Tasks

You are required to implement two C/C++ programs—one using OpenMP and one using MPI—based on the Jacobi iterative method for solving the heat distribution problem in parallel on multiple processors. We will provide the sequential program (`heat_seq.c` / `heat_seq.cpp`) as the baseline. Your task is to develop corresponding OpenMP and MPI versions based on this baseline implementation. Your code should include appropriate comments to improve readability. Any debugging statements must be removed or commented out so that the parallel programs produce exactly the same output as the sequential program. The parallelization effort will center on modifying the `find_steady_state()` function to parallelize the stencil computations.

Part 1: OpenMP Programming

Develop your OpenMP code and save the source file as `heat_mpi.c/cpp`. The sequential program uses nested `for` loops to implement the Jacobi iterative method for computing temperatures. Since OpenMP supports shared-memory parallelism, multiple threads can access the temperature arrays in shared memory. Your task is to add appropriate OpenMP directives to parallelize these loops and improve performance. Carefully determine the suitable work-sharing constructs and data-sharing attributes for all relevant variables. You may experiment with different OpenMP clauses, such as `schedule`, `nowait`, `reduction`, `critical`, `collapse`, etc., to achieve the best-performing implementation.

Part 2: MPI Programming

Develop your MPI code and save the source file as `heat_mpi.c/cpp`. Use row-striped decomposition, as illustrated in the following figure. Each process is responsible for computing a subset of rows of grid points. Points at the edges of a stripe depend on data stored in neighboring processes, so you must exchange the required boundary rows.

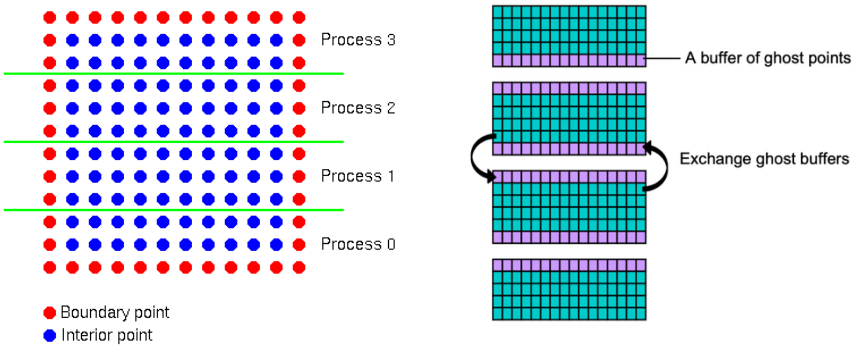


Figure 3. Domain decomposition of a 12x12 mesh on 4 processors (local arrays with ghost cells allocated).

For programming convenience, each process should allocate one additional ghost row at each edge. These ghost points store the boundary row received from neighboring processes, allowing the temperature computations to proceed using a seamless local array.

At the end of execution, your program should output the steady-state temperature array. This requires an additional communication phase to gather the computed results from all processes.

Again, you may experiment different MPI routines, such as standard blocking send/receive, nonblocking send/receive, and collective communication routines to achieve the best-performing implementation.

Notes and Hints

1. You may implement multiple versions for performance experiments but submit only one final version (the best-performing implementation) for your OpenMP program and MPI program.

2. In both the OpenMP and MPI programs, termination is determined by a convergence test based on the maximum temperature difference between successive iterations. If the difference falls below the predefined tolerance ε = 0.001, then all threads or processes must stop further computations and report the result. If the convergence is not reached within the specified maximum number of iterations specified, the program must terminate as a safeguard against infinite execution.

3. For the OpenMP implementation, this maximum difference should be computed correctly across all threads, e.g., updating this difference using an appropriate critical section or reduction. For the MPI implementation, each process should first compute its local maximum difference, and then a global maximum should be obtained across all processes before deciding whether to terminate. This ensures that all OpenMP threads or MPI processes make a consistent termination decision.

4. Although more scalable decomposition schemes exist, such as partitioning the domain into a checkerboard of grid blocks, they typically require more complex communication patterns and may require MPI communicators with a Cartesian topology. For this first assignment, please use row-wise striped partitioning to keep the implementation simple and ensure consistent grading.

5. Other implementation details are left to you, such as whether to use blocking or non-blocking communication.

6. Your parallel program should closely resemble the provided sequential program in both program logic and output. We strongly recommend starting with a copy of the sequential code and modifying it to implement parallelization. Make only the changes necessary for parallelization. The provided code does not validate all inputs or handle all possible errors. For example, it does not reject negative grid dimensions or check for failed `malloc()` calls. Do not add extra validation or error-handling code merely to make the program more robust.

7. One exception to point 6 is that you are required to print the number of OpenMP threads and the number of MPI processes spawned to run the program. Refer to the following sample outputs and follow the output format exactly.

   Sequential Version:

```
Problem size: 200x200
Converged after 30628 iterations with max diff: 0.00099999.
Elapsed time = 2.188142 sec.
Output file heat_seq-200x200.txt created.
```

   OpenMP Version:

```
OpenMP threads: 16
Problem size: 200x200
Converged after 30628 iterations with max diff: 0.00099999.
Elapsed time = 0.462527 sec.
Output file heat_omp-200x200.txt created.
```

   MPI Version:

```
MPI processes: 16
Problem size: 200x200
Converged after 30628 iterations with max diff: 0.00099999.
Elapsed time = 5.793835 sec.
Output file heat_mpi-200x200.txt created.
```

8. Although you may modify the signatures of the provided functions in your parallel programs, keep them unchanged whenever practical to closely resemble the sequential program. For example, the `find_steady_state()` function is surely an exception because you need to pass extra parameters including the MPI process rank and communicator size to support its workload distribution logic. As another example, you may add additional parameters to the `print_solution()` function to help it cooperatively print a global copy of the temperature array to the output file. However, our file system is not a parallel file system; all file I/O operations should be done on one process (presumably rank 0) sequentially. Before file writing takes place, you need to gather the local array contents from all participating processes. This gathering can be done inside the body of `print_solution()` or done in other functions before calling `print_solution()`.

9. You may also reinterpret an existing parameter, e.g., the parameter `m` in `find_steady_state()` originally means the number of rows in the entire temperature array, in your MPI version, it may instead represent the number of rows in the local array (a partition of the global array). Alternatively, you may retain its original meaning and introduce an additional parameter, say `local_m`, to mean the local row count. Clearly document whether the local row count includes ghost rows, and provide appropriate comments explaining any changes to parameter meanings.

10. Do not apply sequential optimizations to the provided code. For example, the sequential program copies the array `w` back to `u` for the next iteration using nested loops that copy the elements individually. Replacing these loops with `memcpy()` calls or buffer swapping (interchanging the `u` and `w` pointers) may reduce execution time. However, these improvements result from sequential optimization rather than parallelization and would make the performance comparison with the sequential baseline unfair. Retain the original element-by-element copying approach and avoid all other sequential optimizations.

11. The count of interior rows may not be divisible by the number of OpenMP threads or MPI processes. You may choose how to distribute any remaining rows, but all interior rows (i.e., all but the top and bottom boundary rows) must be processed. Every participating thread or process must be assigned at least one interior row. Let m be the total number of rows and p the number of threads or processes. You may assume that $m - 2 \geq p$ is always true; no validation of this condition is required.

12. Our correctness checks will primarily compare the console output and output file contents of your parallel programs with those of the sequential program. You should test your programs with a variety of grid sizes. Your parallel programs must retain the functionality of the "live" version: when compiled with the `-DLIVE` option, they must update the output file every 1,000 iterations. When the grid size, `MAX(m, n)`, is smaller than VIEWABLE_SIZE, the temperature array should be printed to the console twice—once before and once after calling `find_steady_state()`—for quick verification, rather than written to a file once.

Performance Evaluation

After development and thorough correctness verification, you will experimentally evaluate and possibly fine-tune your OpenMP and MPI programs using our department's HPC cluster. To ensure fair and accurate time measurements, dedicated compute nodes will be reserved, and execution runs across students will be scheduled via the Slurm workload manager. Detailed guidelines on cluster usage, job submission to Slurm, and conducting performance benchmarking will be covered in a separate tutorial.

Measure the execution times of the provided sequential program and both of your parallel programs, all compiled with the `-O3` optimization flag, using the parameter combinations listed below:

- **Problem size (n)**: 200, 300, 400
- **Number of processors (p)**: 1, 2, 4, 8, 16, 32

Notes:
- The number of processors being used is controlled by the number of threads to run the parallel sections of the OpenMP program or the number of processes to start for the MPI program.
- For the case of p = 1, use the sequential program for measuring the execution time.
- For OpenMP, only one node (server machine) is required because one node has 96 CPU cores.
- For MPI, use the following configuration for your Slurm job submission:

Processes	Nodes	Processes per node
2	1	2
4	1	4
8	2	4
16	4	4
32	4	8

For each combination, derive the **speedup** $S(n,p)$ and **efficiency** $E(n,p)$ as follows:

$$S(n,p) = \frac{T(n,1)}{T(n,p)} \quad \text{...(5)}$$

$$E(n,p) = \frac{S(n,p)}{p} \quad \text{...(6)}$$

where
- $T(n,1)$ is the execution time of the sequential program (`heat_seq.c/cpp`) on one processor for problem size n (i.e., an $n \times n$ grid).
- $T(n,p)$ is the execution time of the parallel program (`heat_omp.c/cpp` or `heat_mpi.c/cpp`) on p processors for problem size n.

The reported execution time should include only the elapsed time of the `find_steady_state()` function. For the OpenMP and MPI programs, use `omp_get_wtime()` and `MPI_Wtime()`, respectively, to measure execution time, rather than the timing method used in the sequential program.

Performance results should be saved in a CSV file of a specific format for our grading and plotted as a chart using our provided Python notebook for ease of visualization and comparison.

The performance charts should report an average execution time because individual runs can vary due to system noise, scheduling, cache effects, network contention, and other runtime fluctuations. Run each program for 5 times for each configuration and use the average execution time for plotting.

Submission Items

Refer to Blackboard for the due date of this assignment and submit the following items to Blackboard > Assignment 1 on time:
- `heat_omp.c` / `heat_omp.cpp`: Source file of the OpenMP C/C++ program
- `heat_mpi.c` / `heat_mpi.cpp`: Source file of the MPI C/C++ program
- `seq_times.csv`: Execution time records for the sequential program
- `omp_times.csv`: Execution time records for the OpenMP program
- `mpi_times.csv`: Execution time records for the MPI program

- `report.ipynb`: A Python notebook reporting the benchmarking results in tabular and graphical formats.
  - Briefly interpret the performance results and relate your explanations to the implementation choices in your parallel programs.
  - There is no fixed word limit, but keep the report concise.

Note: Please follow the above specified filenames (case-sensitive) to facilitate our grading.

Use of Generative AI

Regarding the use of generative AI in assessment, this course adopts ***Approach 3: Use of AI tools is allowed with explicit acknowledgement and proper citation***. If you have used any AI tools to assist in developing your parallel programs, include an acknowledgement, appropriate citations, and the key prompts you used in additional Markdown cells in your report notebook.

Use the following acknowledgement template:

"I acknowledge the use of (name of AI tool and URL) to [describe the assistance received, e.g., obtain advice on improving the performance of my MPI code, debug a race condition in my OpenMP code, check the grammar of my written report, etc.].

Submitting this assignment specification to an AI tool and asking it to generate all the code for the required programs is prohibited. AI tools may assist your working process but must not replace it entirely.